

Research Progress Report

Project Title: Design and Implementation of a Zero Trust Driven Application Trust Establishment Framework in Software Defined Networking.

Problem Statement

Software Defined Networking (SDN) provides centralized control and makes network management more flexible. However, in many existing approaches, once an application is authenticated, authorized, and considered trusted, it can continue communicating with the SDN controller without further trust verification. If the application becomes compromised after the initial trust establishment, it may still gain access to the controller, which can introduce security risks.

To address this limitation, this work focuses on designing and implementing a Zero Trust Driven Application Trust Establishment Framework for SDN. Instead of relying on one-time trust establishment, the proposed framework will verify the trustworthiness of an application every time it requests access to the SDN controller. This follows the Zero Trust principle of "Never Trust, Always Verify" and aims to strengthen the security of the SDN environment.

What I Understood About SDN

While studying SDN, one basic thing became clear first that SDN is different from traditional networking because it separates the control plane from the data plane. In a traditional network, devices make many decisions on their own, but in SDN the network becomes more programmable because control is handled in a more centralized way through the controller. This idea makes network management easier, more flexible, and easier to monitor.

From the basic structure of SDN, I understood that it is mainly explained using three layers.

- (i) The first one is Application Layer, where applications like routing, firewall, monitoring, and load balancing are placed.
- (ii) The second one is the Control Layer, where the SDN controller stays and acts like the brain of the network because it has a global view of the network state.
- (iii) The third one is the Data Layer, which contains switches and forwarding devices that forward packets based on the rules given by the controller.

Another thing I understood is how communication happens inside SDN. Applications communicate with the controller through the northbound interface, and the controller communicates with switches in the data layer through the southbound interface, often using protocols such as OpenFlow. So in a simple way, the application gives a request, the controller takes the decision, and the forwarding devices execute that decision.

After reading the selected SDN paper, it became easier to understand that trust is a very important issue in SDN. The paper explains that network applications are often trusted too easily once they are connected with the controller, even though some of them may be third-party applications. Since these applications can read network state, write policies,

subscribe to events, and access resources, a malicious or badly designed application can create serious security problems inside the network.

What I Understood About ZTA

After studying Zero Trust Architecture, I understood that ZTA follows the idea of "Never Trust, Always Verify." It does not assume that a user or device is safe only because it is already inside the network. Instead, each access request needs to be checked continuously by using different factors such as identity, device condition, behavior, and context.

I also understood that ZTA is not just a single tool or software. It is more like a complete security model that depends on continuous verification, least privilege access, and context-based decision making. This makes it different from traditional trust models, where trust is usually given once and kept for a long time.

Another important understanding is that ZTA includes some major components like Policy Engine, Policy Administrator, and Policy Enforcement Point. These components help in making decisions, applying policies, and enforcing access control. While reading about ZTA, it also became clear that practical implementation is not very simple because issues like scalability, delay, and deployment complexity can appear, especially in cloud, IoT, or programmable network environments.

Difference Between Papers And My Work

The SDN paper mainly focuses on trust management between the SDN controller and network applications. It explains how applications are authenticated, authorized, and trusted before communicating with the controller. However, once an application is marked as trusted, it is generally allowed to continue its communication without further trust verification.

The ZTA paper provides a broader understanding of Zero Trust Architecture and explains the concept of continuous verification. However, it does not specifically discuss how this idea can be applied for application trust establishment in an SDN environment.

My work tries to bring these two ideas together. The proposed framework is integrated with the SDN controller, where every application request passes through the trust establishment module before accessing the controller. Instead of trusting an application after a single successful verification, the framework verifies its trustworthiness every time it requests access. After implementation, the framework has been evaluated by analyzing parameters such as latency, scalability, and network overhead, and the results are compared with the existing SDN environment to understand the impact of continuous trust verification.

Current Project Progress

In the previous report, the work was mostly in the study and understanding phase. Since then, I have completed the actual implementation and evaluation of the proposed framework. The following parts are now done:

- Set up the SDN environment (Ubuntu VM with RYU controller, Mininet, OpenFlow 1.3).
- Designed and developed the Zero Trust Driven Application Trust Establishment Framework.

- Integrated the trust establishment module with the RYU SDN controller so that every application request is verified before it can access the controller.
- Created legitimate application clients and five attack simulation programs.
- Fixed several important bugs that were found during testing.
- Evaluated the framework using detection rate, false positive rate, latency, and scalability.

What I Implemented

The framework is built on top of the RYU SDN controller. The controller works normally as an OpenFlow 1.3 learning switch, but it also exposes a Northbound REST API on port 8080. Every network application must send its requests through this API, so every request goes through the Zero Trust pipeline before any action is taken.

Verification Layers

For every request, the framework runs seven verification steps: token verification, API key validation, device identity verification, Role-Based Access Control (RBAC), Attribute-Based Access Control (ABAC), behavioral analysis, and trust scoring. If any step fails, the request is denied immediately and the switch is not changed.

Trust Scoring and Hybrid Model

Trust is calculated as a composite of two parts. The session trust comes from the verification steps of the current request. The reputation is a long-term score that increases on success and decreases on failure. The final value is computed as 0.7 times the session trust plus 0.3 times the reputation. A trust score above 0.6 is needed for a request to be allowed. The score is updated after every request, so the system follows continuous verification.

Policy Enforcement Point

The Policy Enforcement Point acts as a fail-closed guard. It checks the trust verdict first, and if the request is not allowed, it is denied right away. Only allowed requests are forwarded to the policy engine, and only then is the corresponding OpenFlow action applied to the switch.

Five Attacks and Legitimate Applications

I created legitimate applications with roles such as monitor and admin, and five types of attacks: no credentials, fake device identity, replay, unauthorized REST calls, and flood. This helped to test whether the framework can stop attacks while allowing normal applications to work.

Bugs Found and Fixed During Testing

While testing, I found and fixed several important problems:

- At first, the enforcement point did not actually block behavioral attacks even though the trust verdict said they should be denied. I fixed this by making the enforcement point fail closed, so any request that is not clearly allowed is denied.
- The trust score was not passing correctly to the response, so every result showed trust as 0.0. I fixed this so the reason, matched policy, and trust score are now included in every response.

- All failures were reducing trust by the same amount, which made legitimate applications get locked out. I introduced tiered penalties, so security failures reduce trust more than simple privilege violations.
- The flood detection threshold was too loose. I tightened it so that more than six requests in three seconds is considered a burst, with a progressive penalty that drives a flooding application's trust down to zero.
- There was a timestamp bug in the client that made tokens fail at second boundaries. I fixed it by using the token issue time.

How I Evaluated It

I created a baseline controller that has the same REST interface but no Zero Trust verification, so I could compare the two. The tests were run on the Ubuntu VM with Mininet. I used:

- A detection battery that runs each of the five attack classes 50 times and checks whether they are denied.
- A false positive test that runs the legitimate monitor and admin applications 50 times and checks whether they are ever incorrectly blocked.
- A latency test that measures the time taken per request, both sequential and concurrent, for the baseline and the Zero Trust controller.
- A scalability test that runs the same topology with 4, 8, and 16 hosts.

Results

Attack Detection and False Positives

Test	Result
Attack detection (five classes, 50 runs each)	100% detection rate
Flood request blocking	0.600 (blocks from the 7th request of a burst)
False positives for legitimate monitor	0 out of 50
False positives for legitimate admin	0 out of 50

Latency Overhead Compared to Baseline

Metric	Baseline	Zero Trust	Overhead
Sequential average	4.038 ms	5.520 ms	+1.48 ms (+37%)
Sequential p95	7.352 ms	9.078 ms	+1.73 ms
Concurrent average (10 workers)	25.158 ms	50.181 ms	about 2x

Scalability

Hosts	Ping Loss	Sequential Average	Sequential p95
-------	-----------	--------------------	----------------

4	0.0%	3.612 ms	5.204 ms
8	0.0%	3.804 ms	4.869 ms
16	0.0%	3.660 ms	4.905 ms

The latency stays flat as the number of hosts increases, which means the verification cost depends on each request and not on the size of the topology.

Trust Evolution

From the trust traces, the admin application's trust rises from about 0.88 to 1.00 as it keeps making allowed requests. The monitor's trust drops a little when its boundary probes are denied, but it recovers on its own. The compromised application that floods the controller drops from 0.88 to 0.00, which shows the progressive lockout working.

Challenges Faced

During implementation, understanding how the proposed Zero Trust mechanism can be implemented in a real SDN environment was a challenge. Designing a trust establishment module that verifies every application request before accessing the SDN controller required a lot of practical study.

Setting up the complete SDN environment was also challenging. I had to configure the RYU controller, Mininet, and the proposed framework together on a virtual machine, and repeatedly clean up leftover controller processes before each test so that the measurements were correct.

I also faced challenges in the initial implementation. Some bugs were not visible until the tests were run, such as the behavioral blocking issue and the trust score propagation issue. Finding and fixing these took time but made the framework correct.

Another limitation is that the proposed framework may not be applicable to every SDN deployment, since some organizations may not allow modifications to the existing controller or security infrastructure due to operational policies.

There are also some limitations in the current work. The evaluation was done on Mininet emulation with a single switch, and the concurrent latency is about two times higher because of the eventlet GIL in RYU. The secret keys are currently stored in the code, so a production version would need proper key management.

Upcoming Goals

The next goals of the work are:

- Prepare the complete IEEE-format paper based on the implemented framework and the measured results.
- Compile the paper and finalize the manuscript for submission.
- Review the implementation code and documentation so that every part is clear and reproducible.
- Refine the framework further based on the experimental observations, for example improving the concurrent performance.

Conclusion

From the work carried out so far, it is understood that SDN provides flexibility, programmability, and centralized network management, but it also introduces trust-related security challenges when applications communicate with the SDN controller. The concept of Zero Trust provides a promising approach by ensuring continuous trust verification instead of relying on one-time trust establishment.

The proposed Zero Trust Driven Application Trust Establishment Framework has now been designed, implemented on the RYU SDN controller, and evaluated. The results show that the framework detects all the tested attacks with a low false positive rate for legitimate applications, adds a small and acceptable latency overhead, and keeps its performance flat as the network scales. This confirms that continuous trust verification can strengthen the security of the SDN environment without affecting normal applications.